

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..4

MODULES

 Copy as Markdown

Modules

Modules let you drop ready-made features into your server, such as authentication, or package your own server and app code for reuse across projects and on pub.dev. A module is similar to a Dart package but bundles server code, generated client code, and Flutter widgets, along with its own endpoints and database tables. Each module has its own namespace for endpoints and methods to minimize the risk of conflicts.

The `serverpod_auth_core` and `serverpod_auth_idp` modules, both maintained by the Serverpod team, are examples: they add complete user authentication to your project.

Add a module to your project

A module ships as one package per part of your project, so you add it to each `pubspec.yaml` that uses it.

Add the server package

To add the `serverpod_auth_idp` module, add `serverpod_auth_idp_server` to your server's `pubspec.yaml`:

```
myproject_server/pubspec.yaml
```

```
dependencies:  
  serverpod_auth_idp_server: 4.0.0
```

INFO

Use the same version as your Serverpod packages, such as `4.0.0`. All official packages share one version number, so matching versions work together. Pin the exact version rather than a caret range so the packages stay in sync with the CLI.



Ask AI

In your `config/generator.yaml`, you can optionally list the module and give it a `nickname`, which sets how you [reference the module's models](#). Without an entry, the module's own declared nickname is used; for the official modules that is the full package name.

```
modules:
  serverpod_auth_idp:
    nickname: idp
```

See [Configuration](#) for the full set of `generator.yaml` module options.

Then fetch dependencies; one `dart pub get` anywhere in the workspace resolves every package:

```
dart pub get
```

Start the server, which regenerates your code so the module's endpoints and models become part of your server and client:

```
serverpod start
```

The module adds tables to your database, so create and apply a migration: in the `serverpod start` terminal, press **M**.

Add the client package

In your client's `pubspec.yaml`, add the module's generated client package:

```
myproject_client/pubspec.yaml
```

```
dependencies:
  serverpod_auth_idp_client: 4.0.0
```

Add the Flutter package

In your Flutter app's `pubspec.yaml`, add the module's Flutter package:

```
myproject_flutter/pubspec.yaml
```

```
dependencies:
  serverpod_auth_idp_flutter: 4.0.0
```

Reference a module in your models

You can reference a module's serializable classes from the model files in your project by adding the `module:` prefix, followed by the module's nickname. A module has exactly one effective nickname: the one you set in your `generator.yaml`, or, if you set none, the module's own declared nickname. For the official modules, the declared nickname is the full package name.

With `nickname: auth` set for `serverpod_auth_core` in your `generator.yaml`, reference its `AuthUser` class as:

```
class: MyClass
fields:
  userInfo: module:auth:AuthUser
```

With no `generator.yaml` entry, use the module's declared nickname instead:

```
class: MyClass
fields:
  userInfo: module:serverpod_auth_core:AuthUser
```

Setting your own nickname replaces the declared one, so only one of these forms works at a time.

See [Working with models](#) for how model files work.

Create your own module

With the `serverpod create` command, you can create your own module for code that is shared between projects or that you want to publish to pub.dev. To create a module instead of a server project, pass `module` to the `--template` flag:

```
serverpod create --template module my_module
```

The command creates a server and a client Dart package. If you also want to ship Flutter code, create a package for it with Flutter:

```
flutter create --template package my_module_flutter
```

In the Flutter package, import the client libraries that `serverpod create` generated.

If your module needs database tables, prefix their names with the module name to avoid conflicts with the projects that use it. Serverpod's own tables are prefixed with `serverpod_`.

Related

- [Configuration](#): every `generator.yaml` module option.
- [Authentication](#): the auth modules in practice.
- [serverpod create](#) [reference](#): all create templates and flags.